

Reviewer Pack — Betti Numbers and Resolution Width in 3-CNF

Entry 6bc88662420d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-27 07:04:09 UTC

Betti Numbers and Resolution Width in 3-CNF

Entry ID: 6bc88662420d **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-27 07:04:09 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Topology **Field B** (complexity object): Resolution Width
Statement:

For a random 3-CNF instance with n variables, the first Betti number of the clause-induced simplicial complex is $\Theta(\log n)$ if and only if the resolution width is $\Theta(\log n)$.

Rationale (proposer’s reasoning):

Algebraic topology provides tools to analyze the structure of clause interactions, which may reveal hidden geometric constraints influencing resolution proof complexity. Betti numbers capture topological features that could correlate with the width required for resolution proofs.

Taxonomy category: AVG_T0_WORST_CASE (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 564244dc235496c7

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For each n in $\{8,10,12,14,16,18,20\}$, generate 20 random 3-CNF seeds at clause-to-variable ratio 4.26. Compute first Betti number b_1 and resolution width w . Fit log-log slopes a_{b_1} , a_w of mean values vs $\log(n)$. Conjecture is SUPPORTED iff both slopes lie in $[0.5,1.5]$ and per-instance Pearson correlation $r(b_1,w) \geq 0.7$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.90	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.92	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.90	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - random 3-CNF resolution width simplicial complex Betti - clause hypergraph homology resolution complexity satisfiability - topological invariants proof complexity width lower bound CNF

Top relevant hits considered: - [<http://arxiv.org/abs/1009.4130v3>] Limit theorems for Betti numbers of random simplicial complexes - [<http://arxiv.org/abs/2003.08066v2>] Law of large numbers for Betti numbers of homogeneous and spatially independent random simplicial complexes - [<http://arxiv.org/abs/1501.03759v1>] Erratum: Limit theorems for Betti numbers of random simplicial complexes - [<http://arxiv.org/abs/2110.02458v3>] Magnitude homology of graphs and discrete Morse theory on Asao-Izumihara complexes - [<http://arxiv.org/abs/2306.01534v3>] The magnitude homology of a hypergraph - [<http://arxiv.org/abs/1909.00234v1>] The spectrum of a class of uniform hypergraphs - [<http://arxiv.org/abs/1110.6876v4>] Lower bounds for topological complexity - [<http://arxiv.org/abs/1606.05050v1>] Proof Complexity Lower Bounds from Algebraic Circuit Complexity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from collections import defaultdict

def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        # Find pivot
        max_row = i
        for r in range(i+1, n):
            if abs(A[r][i]) > abs(A[max_row][i]):
                max_row = r
        A[i], A[max_row] = A[max_row], A[i]

        # Eliminate below the pivot
        factor = -A[i][i]
        for k in range(i, n):
            A[i][k] /= factor
        for r in range(i+1, n):
            factor = A[r][i]
            for k in range(i, n):
                if i == k:
                    A[r][k] = 0
                else:
                    A[r][k] += factor * A[i][k]
```

```

def betti_number(clause_set, variable_set):
    n = len(variable_set)
    m = len(clause_set)

    incidence_matrix = [[0 for _ in range(n)] for _ in range(m)]
    for j, clause in enumerate(clause_set):
        for var in clause:
            incidence_matrix[j][variable_set.index(var)] = 1

    gaussian_elimination(incidence_matrix)

    rank = sum(1 for row in incidence_matrix if any(row))
    return rank - 1

def resolution_width(clause_set, variable_set):
    n = len(variable_set)
    m = len(clause_set)

    def dpll(clauses, assignment):
        if not clauses:
            return True
        unit_clause = next((c for c in clauses if sum(1 for v in c if v in assignment) == 0 and sum(1 for v in c if v not in assignment) == 1)), None
        if unit_clause:
            literal = next(v for v in unit_clause if v not in assignment)
            return dpll([c for c in clauses if literal not in c and -literal not in c], assignment + [literal])
        pure_literal = next((v for v in variable_set if sum(1 for c in clauses if v in c) == 0), None)
        if pure_literal:
            return dpll(clauses, assignment + [pure_literal])
        literal = random.choice(variable_set)
        return dpll(clauses, assignment + [literal]) or dpll(clauses, assignment + [-literal])

    max_width = 0
    for _ in range(100):
        assignment = []
        width = 0
        while not dpll(clause_set, assignment):
            literal = random.choice(variable_set)
            if literal not in assignment and -literal not in assignment:
                assignment.append(literal)
                width += 1
        max_width = max(max_width, width)

    return max_width

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [8, 10, 12, 14, 16, 18, 20]
    results = []

    for n in n_values:
        clause_set = set()
        variable_set = set()

        for _ in range(int(4.26 * n)):

```

```

        variables = random.sample(variable_set, 3)
        clause = tuple(sorted(variables))
        clause_set.add(clause)
        for var in variables:
            variable_set.add(var)

    b1 = betti_number(clause_set, variable_set)
    w = resolution_width(clause_set, variable_set)

    results.append({
        "n": n,
        "b1": b1,
        "w": w
    })

if not results:
    return {
        "metric_name": "Betti Number and Resolution Width",
        "metric_value": None,
        "instances_tested": 0,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

log_n_values = [math.log(n) for n in n_values]
log_b1_values = [math.log(result["b1"]) for result in results]
log_w_values = [math.log(result["w"]) for result in results]

a_b1, a_w = (sum(log_b1 * ln_n for log_b1, ln_n in zip(log_b1_values, log_n_values)) / sum(ln_n
            sum(log_w * ln_n for log_w, ln_n in zip(log_w_values, log_n_values)) / sum(ln_n**2

r = sum((log_b1 - a_b1 * ln_n) * (log_w - a_w * ln_n) for log_b1, log_w, ln_n in zip(log_b1_va
        math.sqrt(sum((log_b1 - a_b1 * ln_n)**2 for log_b1, ln_n in zip(log_b1_values, log_n_value

    return {
        "metric_name": "Betti Number and Resolution Width",
        "metric_value": {"a_b1": a_b1, "a_w": a_w},
        "instances_tested": len(results),
        "conjecture_holds": 0.5 <= a_b1 <= 1.5 and 0.5 <= a_w <= 1.5 and r >= 0.7,
        "counterexample": ""
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [11, 23, 37, 53, 71]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    if all("conjecture_holds" in r and r["conjecture_holds"] for r in results):
        mean_a_b1 = sum(r["metric_value"]["a_b1"] for r in results) / len(results)
        mean_a_w = sum(r["metric_value"]["a_w"] for r in results) / len(results)
        support_fraction = len([r for r in results if "conjecture_holds" in r and r["conjecture_ho
        print(f"RESULT: SUPPORTED mean_a_b1={mean_a_b1} mean_a_w={mean_a_w} support_fraction={supp

```

```

elif any("conjecture_holds" in r and not r["conjecture_holds"] for r in results):
    first_failing_seed = next(r["seed"] for r in results if "conjecture_holds" in r and not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE no seeds tested")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5ea9db14.py", line 142, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5ea9db14.py", line 95, in run_trial
    variables = random.sample(variable_set, 3)
                  ~~~~~
  File "/usr/lib/python3.12/random.py", line 413, in sample
    raise TypeError("Population must be a sequence. ")
TypeError: Population must be a sequence. For dicts or sets, use sorted(d).

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a `TypeError` (`random.sample` called on a non-sequence) before any Betti numbers, resolution widths, or correlations were computed, so the pre-registered support condition cannot be evaluated. | next: Fix the bug by passing a sorted list (e.g., `random.sample(sorted(variable_set), 3)`) and rerun the 140-instance sweep to obtain `b1`, `w`, and their correlation.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	423338
2	propose	ollama_remote	qwen3:8b	0	0	396423
3	propose	ollama_remote	qwen3:8b	0	0	432733
4	propose	ollama_remote	qwen3:8b	0	0	409425
5	preregistration	claude_max	opus	0	0	10120
6	novelty	claude_max	opus	0	0	3984
7	novelty	claude_max	opus	0	0	6741

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20002
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17397
10	judge	claude_max	opus	0	0	4879

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 1725042 ms total latency. Provider mix: {'ollama_remote': 6, 'claude_max': 4}

(full prompt+response transcripts available in research/audit/6bc88662420d.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/6bc88662420d.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/6bc88662420d.tar.gz (if generated)